

# 1. INTRODUCTION

## 1.1 Project Overview

The project “Implement Client Script & UI Policy (Incident)” is a ServiceNow micro project focused on improving data entry and validation in Incident records. The implementation uses client-side controls so that field behavior changes dynamically according to Incident conditions.

The project contains one UI Policy and UI Policy Actions, together with three types of Client Scripts: onChange, onSubmit and onCellEdit. These controls work together to make Incident records more complete and consistent.

## 1.2 Purpose

The purpose is to demonstrate how ServiceNow can dynamically make fields mandatory, control read-only behavior, automatically populate values and prevent invalid record submission. The configuration is intended to reduce incomplete or inconsistent Incident data.

# 2. IDEATION PHASE

## 2.1 Problem Statement

Incident records often depend on manual data entry. Incomplete or incorrect values can affect triage, routing, reporting accuracy, SLA compliance and service quality. The project addresses this by enforcing conditional field behavior and validation at the user interface level.

## 2.2 Empathy Map Canvas

The following project-specific empathy map is a design interpretation based on the Incident entry workflow described in the project.

| Area   | User Perspective                                                                               |
|--------|------------------------------------------------------------------------------------------------|
| Says   | “I need to enter the Incident correctly and save it without validation errors.”                |
| Thinks | The correct assignment and urgency information should be available for a high-impact Incident. |
| Does   | Creates or updates an Incident, changes Impact, enters Assigned To and submits the form.       |
| Feels  | Needs a clear and predictable form that indicates what is required.                            |
| Needs  | Automatic field behavior, validation messages and protection against invalid list edits.       |

## 2.3 Brainstorming

- Make Assignment group mandatory for High Impact incidents.
- Control Urgency when Impact is High.
- Automatically set Urgency to High when Impact changes to High.
- Prevent saving when Impact is High and Assigned To is missing.
- Prevent direct State modification through list editing.
- Allow normal State changes through the Incident form.
- Reverse conditional behavior when Impact is changed away from High.

# 3. REQUIREMENT ANALYSIS

### 3.1 Customer Journey Map

| Stage            | User Action                           | System Behavior                        | Expected Outcome                 |
|------------------|---------------------------------------|----------------------------------------|----------------------------------|
| Create Incident  | Open Incident form                    | Form loads with Incident fields        | User can enter data              |
| Set Impact       | Select High                           | UI Policy and onChange script activate | Required/control rules apply     |
| Enter Assignment | Choose Assignment group / Assigned To | Mandatory validation is enforced       | Required information is provided |
| Submit           | Click Submit                          | onSubmit validation runs               | Invalid record is blocked        |
| List Edit        | Edit State in list                    | onCellEdit blocks the change           | State remains unchanged          |
| Form Update      | Change State on form                  | Normal form update is allowed          | Valid State change is saved      |

### 3.2 Solution Requirement

| ID  | Requirement                | Description                                                        |
|-----|----------------------------|--------------------------------------------------------------------|
| FR1 | High Impact UI Policy      | Apply when Impact = 1 – High.                                      |
| FR2 | Assignment group mandatory | Make Assignment group mandatory under the High Impact policy.      |
| FR3 | Urgency read-only          | Make Urgency read-only when Impact is High.                        |
| FR4 | Urgency auto-set           | Set Urgency to High when Impact changes to High.                   |
| FR5 | Save validation            | Prevent save when Impact is High and Assigned To is empty.         |
| FR6 | List edit restriction      | Prevent direct State changes using list editing.                   |
| FR7 | Reverse behavior           | Restore controlled fields when the High Impact condition is false. |

### 3.3 Data Flow Diagram

```

User → Incident Form → Impact Field
↓
UI Policy / onChange
■ ■
Assignment group Urgency
↓ ↓
Mandatory check Auto-set / Read-only
■ ■
Incident Validation
↓
onSubmit Check
■ ■
Reject Save

Incident List → State List Edit → onCellEdit → Block Change
Incident Form → State Update → Normal Form Save → Allow Change

```

### 3.4 Technology Stack

| Technology / Area | Use in Project                  |
|-------------------|---------------------------------|
| Platform          | ServiceNow                      |
| Application Area  | Incident Management             |
| Configuration     | UI Policy and UI Policy Actions |



## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

| Step | Activity                   | Description                                                              |
|------|----------------------------|--------------------------------------------------------------------------|
| 1    | Requirement identification | Understand Incident data-entry and validation needs.                     |
| 2    | UI Policy setup            | Create High Impact Control policy and mandatory Assignment group action. |
| 3    | Urgency control            | Create UI Policy Action to make Urgency read-only.                       |
| 4    | onChange script            | Automatically set Urgency to High for High Impact.                       |
| 5    | onSubmit script            | Validate Assigned To before saving.                                      |
| 6    | onCellEdit script          | Block direct State changes in list view.                                 |
| 7    | Testing                    | Execute mandatory, save, reverse, list-edit and form-update tests.       |
| 8    | Documentation              | Record configuration, source code and test results.                      |

## 6. FUNCTIONAL AND PERFORMANCE TESTING

### 6.1 Performance Testing

The project is a lightweight client-side configuration. The supplied project specification focuses on functional behavior rather than numerical performance benchmarks. Therefore, no measured response time or load-test dataset is claimed in this report. Performance is considered from the expected lightweight execution of client-side field controls.

| ID   | Test                       | Input / Action                           | Expected Result                                  |
|------|----------------------------|------------------------------------------|--------------------------------------------------|
| TC01 | High Impact Mandatory Test | Impact = High; Assignment group empty    | Assignment group is mandatory.                   |
| TC02 | Urgency Read-only Test     | Impact = High                            | Urgency becomes read-only.                       |
| TC03 | Urgency Auto-set Test      | Change Impact to High                    | Urgency is set to High and info message appears. |
| TC04 | Save Validation Test       | Impact = High; Assigned To empty; Submit | Record is not saved and error is displayed.      |
| TC05 | Successful Save Test       | Fill Assigned To and submit              | Incident saves successfully.                     |
| TC06 | Reverse Condition Test     | Change Impact from High to Medium        | Mandatory/read-only policy behavior is reversed. |
| TC07 | List Edit Test             | Edit State directly in Incident list     | Alert appears and change is blocked.             |
| TC08 | Form State Update Test     | Change State from Incident form          | State update is allowed and saved.               |

## 7. RESULTS

The expected project results are based on the configuration and verification steps specified in the project document. The Incident form should dynamically respond to Impact changes, enforce required information, prevent invalid saves, block direct State list edits and allow valid form-based updates.

### 7.1 Output Screenshots

#### [ Figure 7.1: High Impact Control UI Policy configuration ]

Insert screenshot of the UI Policy record.

#### [ Figure 7.2: Assignment group mandatory UI Policy Action ]

Insert screenshot of the UI Policy Action.

#### **[ Figure 7.3: Urgency read-only UI Policy Action ]**

Insert screenshot of the Urgency action.

#### **[ Figure 7.4: onChange Client Script ]**

Insert screenshot showing the script and configuration.

#### **[ Figure 7.5: onSubmit Client Script ]**

Insert screenshot showing the validation script.

#### **[ Figure 7.6: onCellEdit Client Script ]**

Insert screenshot showing the list-edit blocking script.

#### **[ Figure 7.7: High Impact Incident form ]**

Insert screenshot showing Impact = High and controlled fields.

#### **[ Figure 7.8: Validation error ]**

Insert screenshot of the Assigned To validation message.

#### **[ Figure 7.9: List edit blocked ]**

Insert screenshot of the State edit alert.

#### **[ Figure 7.10: Successful form update ]**

Insert screenshot showing a successful Incident update.

## **8. ADVANTAGES & DISADVANTAGES**

### **Advantages**

- Improves consistency of Incident data entry.
- Provides immediate user-interface feedback.
- Automatically sets Urgency for High Impact incidents.
- Reduces incomplete submissions through validation.
- Protects State from direct list editing.
- Keeps the implementation lightweight and focused on Incident management.

### **Disadvantages**

- The implementation is focused on client-side behavior.

- The project covers a specific set of Incident rules rather than all possible Incident workflows.
- The supplied project specification does not include numerical performance measurements.
- The configuration depends on the ServiceNow Incident form and field names used in the project.

## 9. CONCLUSION

The “Implement Client Script & UI Policy (Incident)” project demonstrates the use of ServiceNow client-side controls to improve Incident data quality and user interaction. The implementation combines UI Policies, UI Policy Actions and Client Scripts to control mandatory fields, read-only behavior, automatic value setting and save-time validation. The onCellEdit script also demonstrates how direct list editing can be restricted while normal form-based updates remain available. Overall, the project provides a practical and lightweight example of enforcing business rules at the Incident user interface level.

## 10. FUTURE SCOPE

- Extend conditional controls to additional Incident fields.
- Add more validation rules for different Impact, Urgency and State combinations.
- Integrate notifications or workflow actions for high-impact Incidents.
- Add role-based controls for specialized Incident operations.
- Extend testing with measurable response-time and larger-volume scenarios.
- Integrate the configuration with broader Incident workflow and reporting requirements.

## 11. APPENDIX

### A. Source Code

#### A.1 onChange Client Script – Auto set urgency for high impact

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue == '') {
    return;
  }

  if (newValue == '1') {
    g_form.setValue('urgency', '1');
    g_form.addInfoMessage(
      'Urgency set to High for High impact incident.'
    );
  }
}
```

#### A.2 onSubmit Client Script – Prevent save if Assigned To missing

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
    g_form.showErrorBox(
      'assigned_to',
```

'Assigned To is mandatory for High impact incidents.'

);

return false;

}

return true;

}

### ***A.3 onCellEdit Client Script – Prevent State change via list edit***

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
```

```
  alert('State cannot be updated using list editing. Please open the Incident.');
```

```
  callback(false);
```

```
}
```

### ***A.4 UI Policy Configuration Summary***

Configuration

Value

Purpose

UI Policy Name

High Impact Control

Controls Incident behavior for High Impact.

Table

Incident

Applies to Incident records.

Active

True

Enables the policy.

Condition

Impact is 1 – High

Triggers the policy.

Assignment group

Mandatory

Requires assignment group for High Impact.

Reverse if false

True